IFJC4002 MC Practical Artificial Intelligence with Python

Faculty of Intelligent Computing, Universiti Malaysia Perlis

MACHINE FAILURE PREDICTION
FOR PREDICTIVE MAINTENANCE

Machine Learning Prediction Project - Case 7 (Binary Classification)

Group: MADE IN THAILAND (Walailak)

No.	Name	Student ID
1	Wuttichok Samathi	67115675
2	Watsana Onbut	67128256
3	Sunhanat Thongon	67123232
4	Anas Waekaji	67134155
5	Ratthanan Yidnorradin	67106716

Course coordinator: Ir. Dr. Marni Azira Markom

Submission: October 2026

Streamlit application: https://machine-failure-prediction-icjgu7utmcxqruahaqmzft.streamlit.app/

GitHub repository: https://github.com/WUTTICHOK/Machine-failure-prediction

Abstract

Unplanned machine breakdowns stop production and cost more to repair than planned maintenance. This project develops a machine-learning system that predicts whether a machine will fail during its next operating shift from seven readings taken at inspection: air and process temperature, rotational speed, torque, tool wear, vibration and machine age. The lecturer-provided dataset contains 2,000 simulated machines, of which 319 (16.0%) fail, and 32 records with a missing temperature reading.

A single scikit-learn pipeline performs median imputation, physically motivated feature engineering and scaling, fitted on training data only. Four models were compared on a stratified 60/20/20 train/validation/test split: a majority-class baseline, logistic regression, a decision tree and a random forest. Hyper-parameters were tuned by 5-fold cross-validation, and the decision threshold and final model were chosen on the validation set. **Logistic regression (threshold 0.70) was selected** (validation F1 0.611). On the untouched test set it reaches 85.3% accuracy, 62.5% recall and 53.3% precision for the Failure class (F1 0.576, ROC-AUC 0.884, PR-AUC 0.614), against an F1 of 0 for the baseline. Vibration, torque and tool wear are the strongest predictors.

The model is deployed as a Streamlit dashboard through GitHub and Streamlit Community Cloud. The dashboard validates inputs, accepts a missing temperature reading, reproduces the training predictions exactly on all 15 reference cases, and logs every prediction with automatic review triggers for drift.

Keywords: predictive maintenance, machine failure, logistic regression, random forest, class imbalance, Streamlit, model monitoring

Table of Contents

1. Introduction 5
1.1 Background 5
1.2 Problem statement and intended users 5
1.3 Objectives 5
1.4 Scope 5
2. Dataset and Exploratory Analysis (CO1) 6
2.1 Data source and description 6
2.2 Data quality 6
2.3 Target and class balance 6
2.4 Relationship between inputs and failure 7
2.5 Implications and suitable AI approaches 8
3. Solution Design and Preprocessing (CO2) 9
3.1 Workflow 9
3.2 Feature selection and engineering 9
3.3 Missing-value treatment 9
3.4 Encoding and scaling 10
3.5 Data split and leakage control 10
3.6 Evaluation design 10
4. Model Development and Comparison (CO2) 11
4.1 Models and settings 11
4.2 Results 11
4.3 Interpretation 13
4.4 Error analysis 13
4.5 Feature importance 14
5. Final Model Selection, Limitations and Improvements (CO2) 15
5.1 Selection 15
5.2 Limitations 15
5.3 Improvements linked to the findings 15
6. Implementation, Deployment, Testing and Monitoring (CO3) 17
6.1 Implementation 17
6.2 Dashboard 17
6.3 Input validation 18
6.4 Deployment 19
6.5 Testing 19
6.6 Monitoring 20
7. Conclusion 22
References 23
Appendix A: Contribution Record 24
Appendix B: AI-Assisted Development Record 25
Appendix C: Test Log and Monitoring Record 26
Appendix D: Dashboard Screenshots 27
Appendix E: Additional Exploratory Figures 31

List of Figures

Figure 1: Class balance of the target (left) and missing values per input (right) 7
Figure 2: Failure rate by quintile of each input (dashed line = overall rate of 16%) 7
Figure 3: Correlation matrix of the inputs and engineered features 8
Figure 4: Model development workflow 9
Figure 5: Test-set confusion matrices of the four models at their tuned thresholds 12
Figure 6: Precision-recall curves on the test set (dashed line = baseline prevalence of 16%) 12
Figure 7: Permutation importance of the selected model (drop in test PR-AUC when an input is shuffled) 14
Figure 8: Predict tab: a high-risk machine is classified as FAILURE (probability 99.4%) 18
Figure 9: Batch tab: three invalid rows are rejected with a reason; a missing temperature and an out-of-range speed are accepted with warnings 19
Figure 10: Monitoring tab: review triggers fire after 40 high-risk machines are logged 21
Figure 11: Predict tab: a typical machine is classified as NO FAILURE (probability 20.8%) 27
Figure 12: Predict tab: speed above the training range and a missing air temperature produce warnings 28
Figure 13: Notebook agreement tab: all 15 reference cases match the training script 29
Figure 14: About tab: data, inputs, split, selected model and limitations 30
Figure 15: GitHub repository used for deployment 30
Figure 16: Distribution of each input and engineered feature by class 31
Figure 17: Box plots of the inputs by class 31

List of Tables

Table 1: Dataset columns 6
Table 2: Data quality checks 6
Table 3: Mean of each input by class and point-biserial correlation with the target 7
Table 4: Candidate AI approaches 8
Table 5: Engineered features 9
Table 6: Models, settings and tuned hyper-parameters 11
Table 7: Cross-validation and validation results (selection data) 11
Table 8: Test-set results (400 machines: 64 Failure, 336 No Failure) 11
Table 9: Test-set outcomes of the logistic regression by tool wear 13
Table 10: Mean inputs by outcome type (test set) 13
Table 11: Standardised logistic-regression coefficients (positive = higher failure risk) 14
Table 12: Selection against practical constraints 15
Table 13: Improvements linked to the findings 15
Table 14: Project files 17
Table 15: Input validation rules 18
Table 16: Examples of tests on the deployed dashboard 19
Table 17: Contribution of each member 24
Table 18: Representative AI-assisted prompts and verification 25
Table 19: Test log of the deployed dashboard 26

1. Introduction

1.1 Background

Predictive maintenance uses condition data from machines to schedule maintenance before a failure occurs, instead of reacting after a breakdown or replacing parts on a fixed calendar. Machine-learning classifiers are a common tool for this task because they can combine many sensor readings into a single risk estimate.

1.2 Problem statement and intended users

The intended users are **maintenance planners and machine operators**. At each inspection they record the condition of a machine and must decide whether it can run the next shift or should be inspected first. A missed failure (false negative) causes an unplanned stop, a damaged workpiece and an expensive repair; a false alarm (false positive) costs only an extra inspection. The system therefore estimates, for each machine, the probability of failure in the next shift and flags machines whose risk exceeds a decision threshold.

1.3 Objectives

1. Analyse the dataset, its quality and the relationship between inspection readings and failure (CO1).
2. Design a leakage-free preprocessing pipeline and compare at least two suitable algorithms against a baseline (CO2).
3. Select a final model using validation results and practical constraints, and explain its limitations (CO2).
4. Implement, test and deploy an interactive Streamlit dashboard with input validation and monitoring (CO3).

1.4 Scope

The task is binary classification on tabular data (Failure / No Failure in the next shift). The data are synthetic teaching data provided by the lecturer; no real machines, people or outcomes are involved, so the results demonstrate the method but are not evidence of real-world accuracy.

2. Dataset and Exploratory Analysis (CO1)

2.1 Data source and description

The dataset 07_Machine_Failure.xlsx (sheet Data) was provided by the course lecturer. Each of the 2,000 rows is one simulated machine at an inspection time, and each machine occurs once. Table 1 describes the columns.

Table 1: Dataset columns

Column	Type	Unit / labels	Role	Meaning
record_id	Text	ID	Identifier (excluded)	Unique synthetic record identifier
air_temperature_c	Decimal	°C	Input	Ambient temperature at inspection
process_temperature_c	Decimal	°C	Input	Process temperature at inspection
rotational_speed_rpm	Integer	rpm	Input	Measured rotational speed
torque_nm	Decimal	N·m	Input	Measured torque
tool_wear_min	Integer	minutes	Input	Accumulated cutting-tool use
vibration_mm_s	Decimal	mm/s	Input	Vibration velocity before the prediction
machine_age_years	Decimal	years	Input	Machine age, 0.5 to 15 years
failure_next_shift	Category	Failure / No_Failure	Target	Failure during the next operating shift

2.2 Data quality

Table 2: Data quality checks

Check	Result	Treatment
Rows × columns	2,000 × 9	-
Missing values	16 in air_temperature_c, 16 in process_temperature_c (32 rows); target and IDs complete	Median imputation inside the pipeline (Section 3.3)
Duplicate rows / IDs	0 / 0	None needed
Unusual values (1.5 × IQR rule)	1 value in vibration_mm_s; none elsewhere	Kept: physically plausible
Consistency	Process temperature is above air temperature in every complete row	Used as a validation warning in the app

The numeric inputs are spread almost uniformly over their ranges (for example rotational speed 901 to 2,499 rpm, torque 15 to 85 N·m, tool wear 0 to 250 min), with no extreme outliers (Appendix E). Missing values affect only the two temperature columns and only 1.6% of rows, so dropping them would lose little data; however, imputation was chosen because a temperature sensor can also fail in use, and the deployed dashboard must still produce a prediction.

2.3 Target and class balance

The target is imbalanced: 319 machines fail (15.95%) and 1,681 do not (84.05%), as shown in Figure 1. A model that always predicts No Failure is therefore 84% accurate while detecting no failures, so accuracy alone is misleading. Evaluation focuses on precision, recall, F1-score and PR-AUC for the Failure class, and class weighting and threshold tuning are used during modelling.

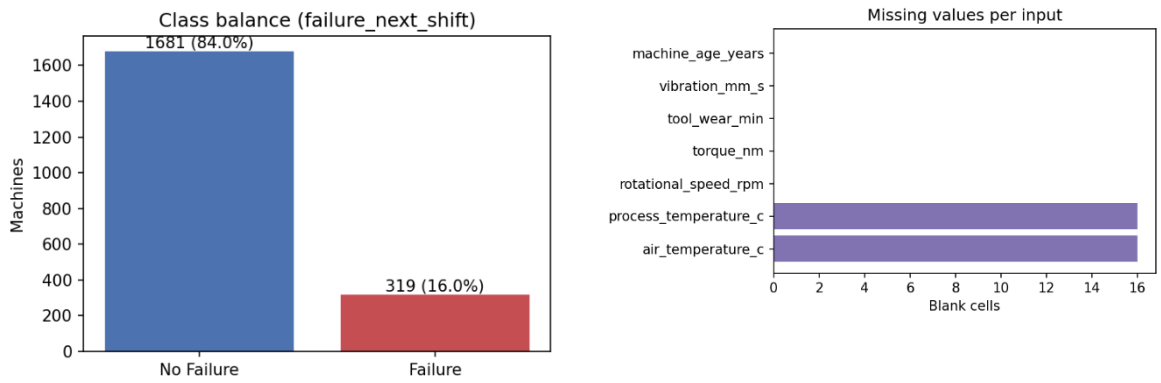


Figure 1: Class balance of the target (left) and missing values per input (right)

2.4 Relationship between inputs and failure

Table 3 compares the average reading of failing and non-failing machines, and Figure 2 shows the failure rate across quintiles of each input.

Table 3: Mean of each input by class and point-biserial correlation with the target

Input	Mean: Failure	Mean: No Failure	Correlation with Failure
Vibration (mm/s)	3.55	2.30	+0.419
Tool wear (min)	186.0	112.5	+0.372
Torque (N·m)	58.7	48.6	+0.182
Machine age (years)	8.84	7.62	+0.106
Process temperature (°C)	73.8	72.4	+0.064
Air temperature (°C)	28.2	27.8	+0.025
Rotational speed (rpm)	1,659	1,710	-0.040
Engineered: tool wear × torque	-	-	+0.417

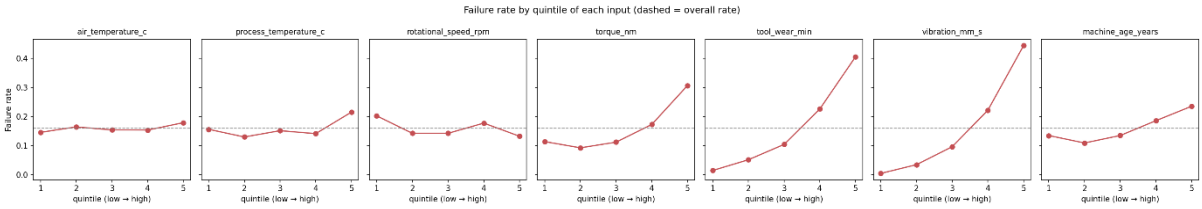


Figure 2: Failure rate by quintile of each input (dashed line = overall rate of 16%)

- **Vibration and tool wear dominate.** The failure rate rises from 0.5% in the lowest vibration quintile to 44.3% in the highest, and from 1.5% to 40.5% across tool-wear quintiles.
- **High torque and age add risk.** Failure rate is about 9 to 11% for low and middle torque but 30.7% in the top torque quintile; machines older than about 12 years fail 23.5% of the time.
- **Temperatures and speed matter little on their own.** Air temperature and speed show no clear trend; only the hottest process-temperature quintile is higher (21.5%).
- **Relationships are smooth and monotonic,** which suits a linear model of the log-odds, and the interaction tool wear × torque is the second most correlated feature.

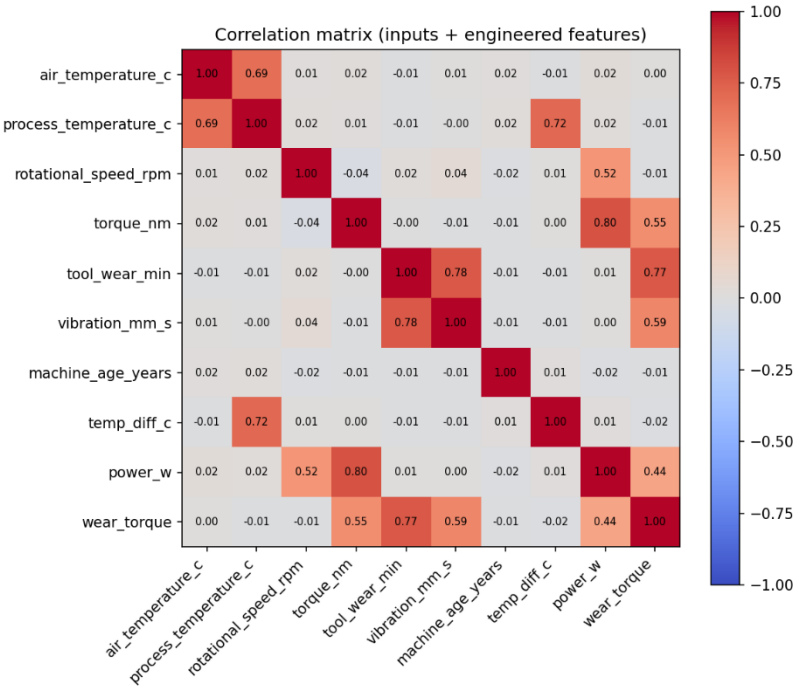


Figure 3: Correlation matrix of the inputs and engineered features

The raw inputs are almost uncorrelated with each other, so there is little redundancy among them. The engineered features are, by construction, correlated with their components (for example tool wear × torque with tool wear and torque), which matters when interpreting linear-model coefficients (Section 4.5).

2.5 Implications and suitable AI approaches

The data are small (2,000 rows), imbalanced (16% failures), almost noise-free in the inputs, and the Guide sheet states that the target is probabilistic: failure probability increases with tool wear, vibration, high torque and thermal stress, but is not a fixed rule. Some failing machines will therefore look normal, which caps achievable recall and precision.

Table 4: Candidate AI approaches

Approach	Why it fits this case	Expected weakness
Logistic regression	Models smoothly increasing risk; interpretable coefficients; calibrated-style probabilities; fast and tiny	Cannot learn sharp thresholds or complex interactions unless engineered
Decision tree	Learns readable rules such as high vibration and high wear leads to failure	Unstable and prone to overfitting with only about 1,200 training rows
Random forest	Averages many trees; captures interactions; robust to scaling and outliers	Larger, slower and less interpretable; can still fit noise in a probabilistic target

3. Solution Design and Preprocessing (CO2)

3.1 Workflow

Figure 4 summarises the workflow implemented in train.py. All preprocessing steps live inside one scikit-learn Pipeline, so they are learned from the training rows only and are saved together with the model for the dashboard.

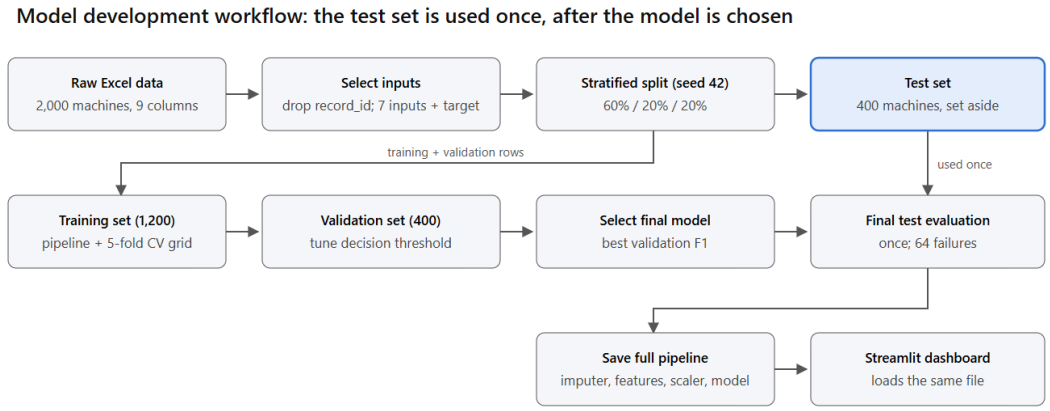


Figure 4: Model development workflow

3.2 Feature selection and engineering

All seven inspection readings are used as inputs because each is available before the shift starts. record_id is excluded because it is an identifier with no causal meaning. The dataset contains no outcome-revealing fields, so no other column had to be removed for leakage. Three features are engineered inside the pipeline from the inputs only:

Table 5: Engineered features

Engineered feature	Formula	Rationale
temp_diff_c	process temperature - air temperature	Thermal stress named in the dataset guide
power_w	torque × rotational speed × 2π / 60	Mechanical load on the machine (W)
wear_torque	tool wear × torque	Load on a worn tool; highest single correlation with failure (0.417)

3.3 Missing-value treatment

A SimpleImputer replaces a missing air or process temperature with the **median of the training set** (27.7 °C and 72.5 °C). The median is robust and appropriate because the missing values are few and appear unrelated to the target. Imputation runs before feature engineering, so the temperature difference is always defined. The target is never imputed. The dashboard lets the user mark a temperature as unavailable and applies the same imputer.

3.4 Encoding and scaling

All inputs are numeric, so no categorical encoding is required; the target labels Failure / No_Failure are mapped to 1 / 0. A StandardScaler standardises the ten model features. Scaling is required for regularised logistic regression and is harmless for tree models.

3.5 Data split and leakage control

Following the dataset guide, the data are split once into stratified **training (60%, 1,200 machines, 191 failures), validation (20%, 400 machines, 64 failures) and test (20%, 400 machines, 64 failures)** sets with seed 42. Each machine occurs once and the data are not a time series, so a stratified random split keeps every machine in exactly one set and no chronological order needs to be preserved.

- The imputer, feature engineering and scaler are fitted inside the pipeline, so cross-validation folds, the validation set and the test set never influence them.
- Hyper-parameters are chosen by 5-fold stratified cross-validation on the training set only.
- The decision threshold and the final model are chosen on the validation set.
- The test set is used once, after all choices are fixed, to give an unbiased final estimate.

3.6 Evaluation design

As required by the brief, every model is evaluated with a confusion matrix, accuracy, precision, recall and F1-score, with **Failure as the positive class and binary averaging**. Macro-F1, ROC-AUC and PR-AUC are added because they are more informative under class imbalance. The decision threshold of each model is the value between 0.05 and 0.95 that maximises F1 on the validation set, because the default 0.5 is rarely optimal when classes are weighted.

4. Model Development and Comparison (CO2)

4.1 Models and settings

Three algorithms of different type and a simple baseline were trained with the same pipeline, split, folds and seed. class_weight makes errors on the rare Failure class more costly during training.

Table 6: Models, settings and tuned hyper-parameters

Model	Fixed settings	Grid searched (5-fold CV, score = F1)	Selected parameters
Baseline	Always predicts the majority class (No Failure)	-	-
Logistic Regression	class_weight = balanced, L2 penalty, max_iter = 5000	C in {0.01, 0.1, 1, 10}	C = 1
Decision Tree	class_weight = balanced	max_depth in {3, 4, 6, 8}; min_samples_leaf in {5, 10, 20}	max_depth = 4, min_samples_leaf = 20
Random Forest	300 trees, class_weight = balanced_subsample	max_depth in {None, 8}; min_samples_leaf in {1, 5}; max_features in {sqrt, 0.5}	max_depth = None, min_samples_leaf = 5, max_features = sqrt

4.2 Results

Table 7 reports cross-validation and validation results, used for selection, and Table 8 reports the final test-set results. Positive class = Failure.

Table 7: Cross-validation and validation results (selection data)

Model	CV F1 (threshold 0.5)	Tuned threshold	Validation F1	Validation recall	Validation precision
Baseline	0.000	0.50	0.000	0.000	0.000
Logistic Regression	**0.526**	**0.70**	**0.611**	**0.688**	**0.550**
Decision Tree	0.491	0.75	0.453	0.484	0.425
Random Forest	0.544	0.39	0.568	0.719	0.469

Table 8: Test-set results (400 machines: 64 Failure, 336 No Failure)

Model	Acc.	Prec.	Recall	F1	Macro F1	ROC AUC	PR AUC	TN	FP	FN	TP
Baseline	0.840	0.000	0.000	0.000	0.457	0.500	0.160	336	0	64	0
Logistic Regression	**0.853**	**0.533**	**0.625**	**0.576**	**0.743**	**0.884**	**0.614**	**301**	**35**	**24**	**40**
Decision Tree	0.785	0.372	0.500	0.427	0.647	0.762	0.377	282	54	32	32
Random Forest	0.800	0.426	0.719	0.535	0.704	0.852	0.593	274	62	18	46

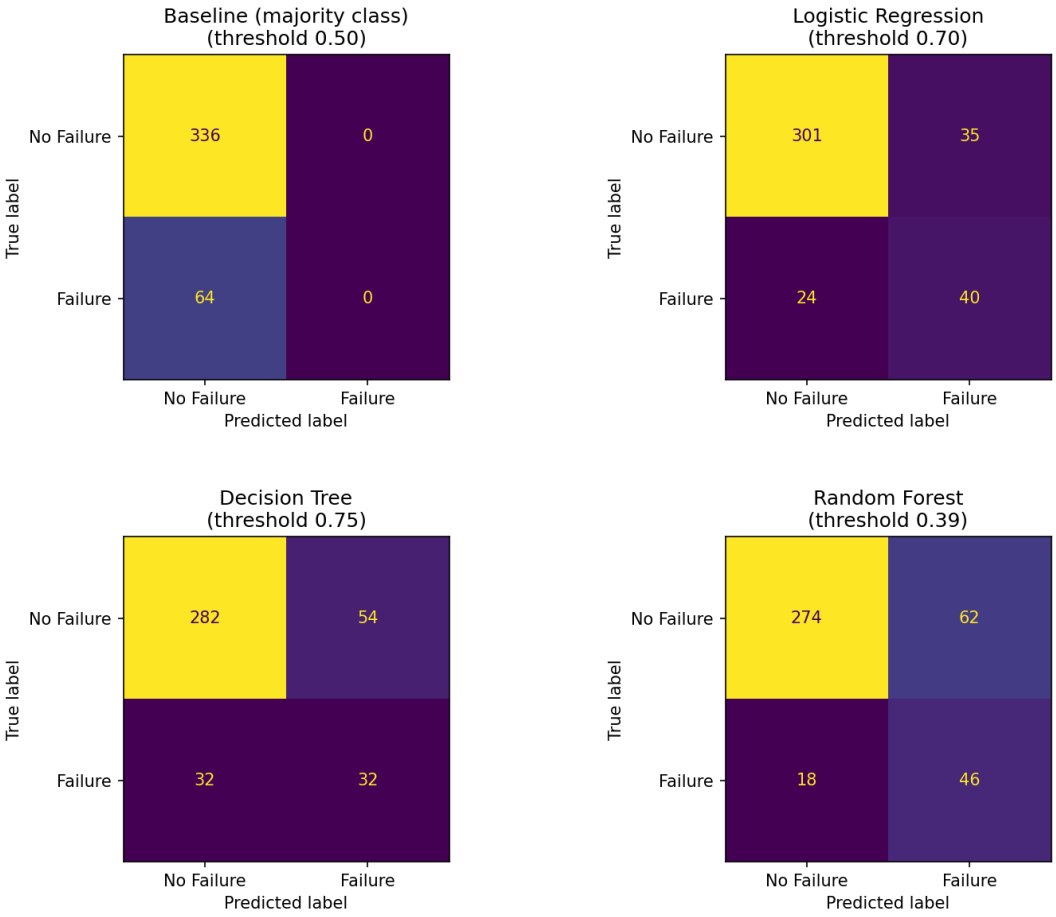


Figure 5: Test-set confusion matrices of the four models at their tuned thresholds

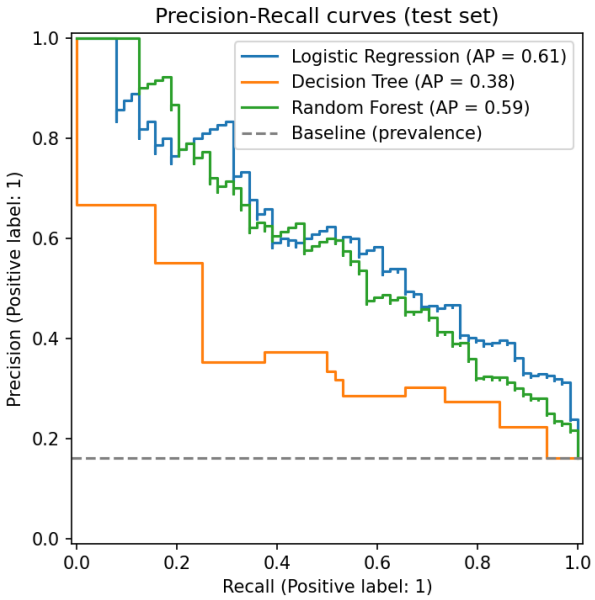


Figure 6: Precision-recall curves on the test set (dashed line = baseline prevalence of 16%)

4.3 Interpretation

- **Baseline.** 84.0% accuracy with no failure detected confirms that accuracy cannot be the selection metric.
- **Logistic regression** gives the best balance: it detects 40 of 64 failures with 35 false alarms, and has the highest ROC-AUC (0.884) and PR-AUC (0.614). Because the dataset guide describes failure risk that rises smoothly with wear, vibration, torque and thermal stress, a model that is linear in the log-odds matches the data-generating process well. Its tuned threshold of 0.70 is high because balanced class weights push predicted probabilities upward.
- **Decision tree** performs worst (test F1 0.427). A single shallow tree divides smoothly rising risk into a few steps and, with a probabilistic target, its splits partly follow noise.
- **Random forest** has the highest recall (0.719, 46 failures detected) but needs a low threshold (0.39) to reach it and produces 62 false alarms, almost twice as many as logistic regression; its precision is only 0.426.
- Validation and test results rank the models in the same order, so the selection is not an artefact of one particular split.

4.4 Error analysis

Errors of the selected logistic regression on the test set were analysed by tool wear (Table 9) and by the average profile of each outcome type (Table 10).

Table 9: Test-set outcomes of the logistic regression by tool wear

Tool wear (min)	Machines	Failures	Detected	Missed	False alarms
0 - 100	161	7	1	6	0
101 - 150	73	7	3	4	3
151 - 200	89	17	8	9	13
201 - 250	77	33	28	5	19

Table 10: Mean inputs by outcome type (test set)

Outcome	Count	Tool wear (min)	Vibration (mm/s)	Torque (N·m)	Age (years)
True positive	40	204.1	3.87	66.0	10.1
False negative (missed failure)	24	152.9	3.11	46.7	9.1
False positive (false alarm)	35	204.9	3.78	57.7	9.6
True negative	301	103.4	2.16	48.1	7.3

- **Missed failures look like healthy machines.** The 24 missed failures have moderate wear (153 min), vibration (3.1 mm/s) and torque (46.7 N·m), close to the true negatives. Because the target is probabilistic, these failures are not predictable from the readings; most of them (10 of 24) occur at tool wear of 150 min or less.
- **False alarms look like real failures.** False positives have almost the same profile as true positives (wear 205 min, vibration 3.8 mm/s), so they are high-risk machines that happened not to fail. In practice, inspecting them is reasonable.
- **Detection is strongest where risk is highest:** 28 of 33 failures (85%) are detected when tool wear exceeds 200 min, at the cost of 19 false alarms in that group.

- **Missing temperatures are handled well:** 7 test machines had a missing temperature, and 6 of them were classified correctly after median imputation.

Practical consequence: a missed failure at low tool wear would lead to an unplanned stop. Operators should keep the regular inspection schedule and use the model to prioritise machines, not to skip inspections.

4.5 Feature importance

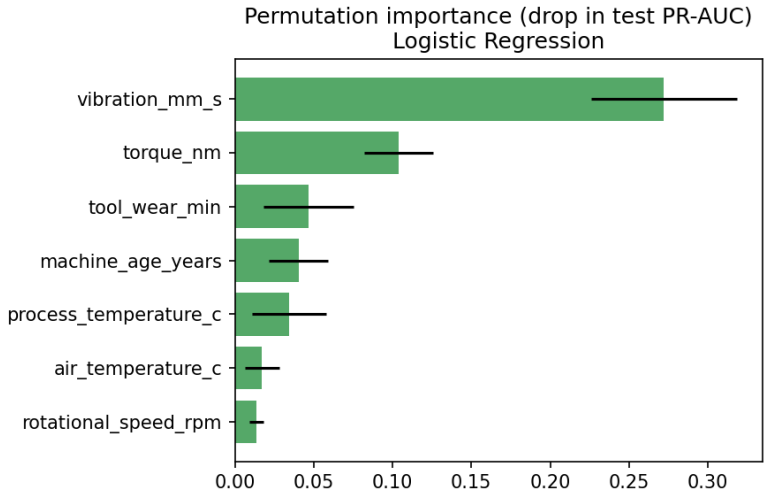


Figure 7: Permutation importance of the selected model (drop in test PR-AUC when an input is shuffled)

Permutation importance (Figure 7) ranks vibration far above the other inputs (PR-AUC drop 0.272), followed by torque (0.104), tool wear (0.047) and machine age (0.041); temperatures and speed contribute little. Table 11 lists the standardised logistic-regression coefficients.

Table 11: Standardised logistic-regression coefficients (positive = higher failure risk)

Feature	Coefficient	Feature	Coefficient
vibration_mm_s	+1.374	temp_diff_c	+0.162
torque_nm	+1.369	process_temperature_c	+0.080
tool_wear_min	+1.092	rotational_speed_rpm	+0.023
machine_age_years	+0.417	air_temperature_c	-0.052
wear_torque	-0.541	power_w	-0.305

Vibration, torque, tool wear and age raise the failure risk, in line with the dataset guide. The negative coefficients of wear_torque and power_w do not mean that load reduces risk: these features are strongly correlated with their components, so the model shares their effect between correlated terms. The combined effect of higher wear and torque remains strongly positive. This collinearity is a known limitation when reading individual coefficients.

5. Final Model Selection, Limitations and Improvements (CO2)

5.1 Selection

The final model is **logistic regression with a decision threshold of 0.70**. It has the highest validation F1 (0.611), and the untouched test set confirms the choice (F1 0.576, recall 0.625, precision 0.533, ROC-AUC 0.884). Table 12 weighs it against the alternatives on practical constraints.

Table 12: Selection against practical constraints

Criterion	Logistic Regression	Decision Tree	Random Forest
Validation F1 / test F1	0.611 / 0.576	0.453 / 0.427	0.568 / 0.535
Test recall / false alarms per 400	0.625 / 35	0.500 / 54	0.719 / 62
Training time incl. grid search	0.6 s	1.8 s	54.8 s
One prediction (measured)	5.0 ms	4.8 ms	82.3 ms
Saved model size	3.3 KB	5.6 KB	2.9 MB
Interpretability	High: coefficients	High: rules	Medium: importance only

Logistic regression is also the fastest, smallest and most interpretable model, which suits a free Streamlit Community Cloud deployment and makes each prediction easy to explain to maintenance staff. If a missed failure were judged much more expensive than an inspection, the random forest at a low threshold would detect 6 more failures per 400 machines but raise 27 more false alarms; with no cost data available, the F1-optimal logistic regression is the defensible choice.

5.2 Limitations

1. **Probabilistic target.** Some failures occur on machines with ordinary readings, so about one in three failures (24 of 64) is missed and precision stays near 53%.
2. **Small test set.** With 64 failing test machines, one more or fewer detection changes recall by 1.6 percentage points; the metrics carry real uncertainty.
3. **Synthetic data.** The data were simulated for teaching; real machines add sensor drift, noise, missing streams and machine-specific behaviour.
4. **Snapshot inputs.** Each machine is observed once, so trends such as rising vibration over several shifts cannot be used.
5. **Collinear engineered features** make individual coefficients harder to interpret (Section 4.5).
6. **Threshold chosen for F1,** not for real maintenance costs, which were not available.
7. **No ground truth after deployment.** Logged predictions cannot show real accuracy until actual outcomes are recorded.

5.3 Improvements linked to the findings

Table 13: Improvements linked to the findings

Finding	Proposed improvement
Missed failures have ordinary readings	Collect repeated measurements per machine and add trend features such as vibration change since the last inspection

Finding	Proposed improvement
Vibration is the strongest predictor	Add richer vibration data (frequency spectrum, peak values) from the same sensor
Threshold maximises F1 only	Set the threshold from the real cost of a missed failure versus an inspection
Collinear engineered features	Use L1 regularisation or drop redundant features to obtain cleaner coefficients
Trees did not beat a linear model	Try gradient boosting with monotonic constraints, and probability calibration
No outcomes after deployment	Record whether each logged machine actually failed and retrain periodically

6. Implementation, Deployment, Testing and Monitoring (CO3)

6.1 Implementation

Table 14: Project files

File	Role
features.py	Single definition of input names, feature order and engineered features, imported by both training and the app
train.py	EDA, split, grid search, threshold tuning, evaluation, error analysis; saves the model and reports; fixed seed 42
predictor.py	Input validation, prediction, logging and monitoring triggers; unit-tested separately from the user interface
app.py	Streamlit dashboard
models/machine_failure_model.joblib	Whole fitted pipeline (imputer, features, scaler, model) plus threshold, feature order and training statistics
tests/test_predictor.py	29 automated tests: normal, boundary, missing, invalid, agreement and monitoring
Case7_Machine_Failure_Colab.ipynb	Reproduces the full workflow in Google Colab, records exact library versions, saves to Google Drive and pushes to GitHub

Because the saved file contains the entire pipeline, the dashboard passes the raw inputs in the order defined by features.py and applies exactly the training transformations. The model was trained with Python 3.13 and scikit-learn 1.6.1, and requirements.txt pins the same versions for deployment.

6.2 Dashboard

The dashboard (Appendix D shows every page) has five tabs. **Predict** offers a number input for each reading, limited to a physically valid range and showing the training range as help text, plus a Reading unavailable option for each temperature. The result is labelled FAILURE or NO FAILURE in the next shift, with the failure probability, the 0.70 threshold and the derived temperature difference, power and wear × torque (Figure 8). **Batch / invalid-input test** validates an uploaded CSV row by row. **Notebook agreement** re-predicts the reference cases saved by train.py. **Monitoring** shows logged predictions and review alerts, and **About** documents the data, model and limitations. The sidebar shows the model and its test metrics; the header and footer show the group logo and name.

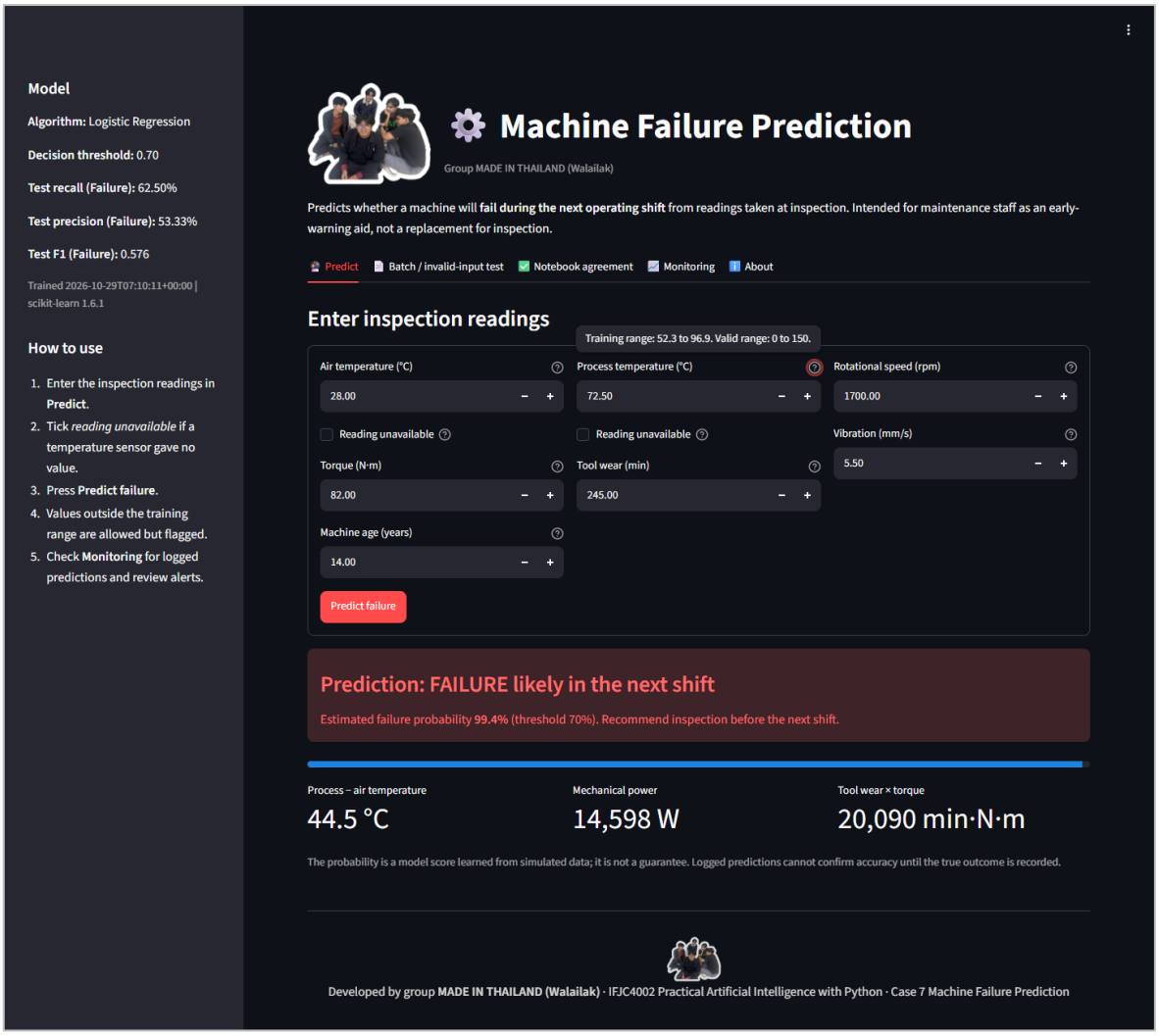


Figure 8: Predict tab: a high-risk machine is classified as FAILURE (probability 99.4%)

6.3 Input validation

Table 15: Input validation rules

Situation	Behaviour
Non-numeric, infinite or missing value (except temperatures)	Rejected with a message naming the field
Outside physical limits (air -20 to 60 °C, process 0 to 150 °C, speed 1 to 5,000 rpm, torque 0 to 150 N·m, wear 0 to 400 min, vibration 0 to 50 mm/s, age 0 to 50 years)	Rejected
Inside physical limits but outside the training range	Predicted, with an extrapolation warning
Missing air or process temperature	Predicted with the training median, with a warning
Process temperature not above air temperature	Predicted, with a warning (never occurs in the training data)

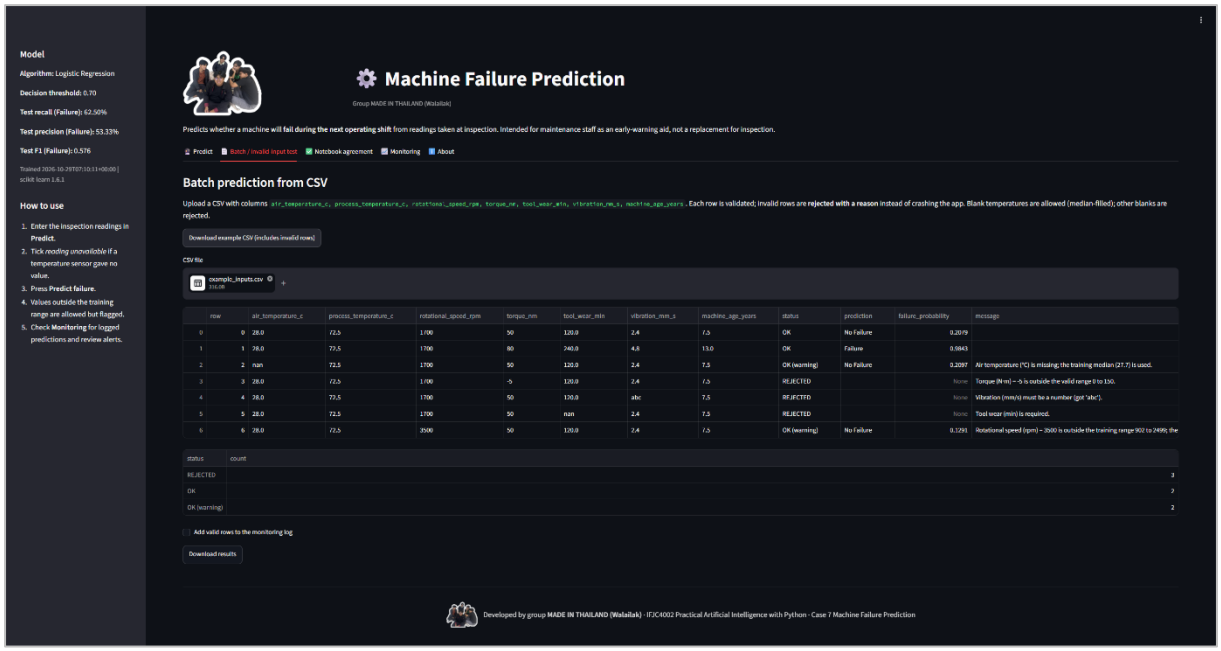


Figure 9: Batch tab: three invalid rows are rejected with a reason; a missing temperature and an out-of-range speed are accepted with warnings

6.4 Deployment

The code, data, trained model and reports are stored in a public GitHub repository and deployed on Streamlit Community Cloud from app.py (Appendix D, Figure 15).

Application: https://machine-failure-prediction-icjgu7utmcxqruahaqmzft.streamlit.app/

Repository: https://github.com/WUTTICHOK/Machine-failure-prediction

A first deployment stalled for more than ten minutes because Streamlit Cloud defaulted to Python 3.14, for which the pinned libraries had no pre-built packages and had to be compiled. Redeploying with the same Python version as the Colab training environment solved the problem. To update the app, the notebook is re-run; its last section pushes the new model and reports to GitHub, Streamlit redeploys automatically, and the agreement tab is checked again.

6.5 Testing

Notebook-dashboard agreement. In the deployed app all **15 reference cases match** the training script (10 test-set machines, 4 boundary cases and 1 case with both temperatures missing); the largest probability difference is 1.1×10^{-16}, which is floating-point rounding (Appendix D, Figure 13).

Functional tests on the deployed app are recorded in Appendix C. All normal, boundary, missing-value and invalid-input cases behaved as expected, for example:

Table 16: Examples of tests on the deployed dashboard

Test	Input	Result
Normal	28 / 72.5 °C, 1,700 rpm, 50 N·m, 120 min, 2.4 mm/s, 7.5 years	NO FAILURE, probability 20.8%
High risk	Torque 82 N·m, wear 245 min, vibration 5.5 mm/s, age 14 years	FAILURE, probability 99.4%

Test	Input	Result
Boundary + missing	Speed 3,500 rpm, air temperature unavailable	Two warnings, prediction made (13.0%)
Invalid	Torque -5; vibration abc; tool wear blank	All three rows rejected with a reason

The automated test suite (tests/test_predictor.py, 29 tests) checks validation, imputation, boundary values, batch rejection, agreement with the training script and both monitoring triggers; it is run in section 6 of the Colab notebook.

6.6 Monitoring

Every prediction is appended to logs/predictions.csv and written to the Streamlit application log. Each record holds only machine readings, the probability, the prediction and warning flags, and no personal data. The Monitoring tab shows, for the most recent predictions, the predicted failure rate, the mean probability, the share of out-of-range inputs and missing temperatures, and the shift of each input mean in training standard deviations. **A review is triggered** when, with at least 30 predictions in the window:

- the predicted failure rate exceeds twice the training failure rate (15.9%), or
- more than 10% of inputs lie outside the training range, or
- more than 10% of inputs have a missing temperature (a possible sensor fault), or
- the mean of any input shifts by more than one training standard deviation.

To demonstrate monitoring, 40 deliberately high-risk machines (high wear and vibration) were submitted through the batch tab. With 63 logged predictions (window of 50), the predicted failure rate rose to 84.0% against 15.9% in training, and mean tool wear and vibration shifted by +1.36 and +1.55 standard deviations. All three triggers fired (Figure 10), correctly signalling that the incoming machines differ from the training population. In real use this would prompt a check of the machine fleet and of recorded outcomes before retraining. Logged predictions alone cannot establish real-world accuracy; true outcomes must be recorded.

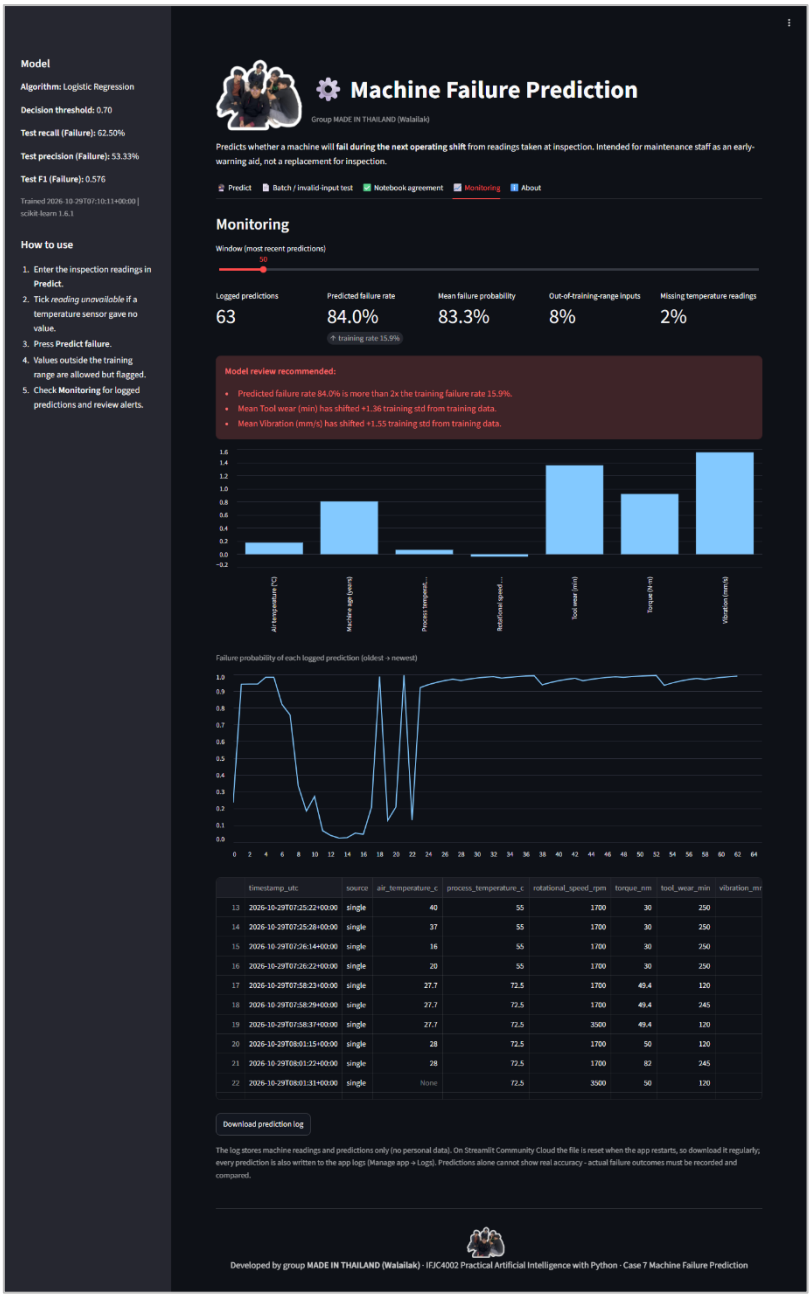


Figure 10: Monitoring tab: review triggers fire after 40 high-risk machines are logged

7. Conclusion

This project built a complete prediction system for machine failure in the next operating shift. Exploratory analysis showed that vibration, tool wear and torque drive failure risk in a smooth, monotonic way, and that 16% of machines fail. After a leakage-free comparison of four models on a stratified train/validation/test split, logistic regression with a threshold of 0.70 was selected. On unseen test data it detects 62.5% of failures with 53.3% precision (F1 0.576, ROC-AUC 0.884), clearly above the baseline, while being the fastest, smallest and most interpretable model. Missed failures mostly occur on machines with ordinary readings, which reflects the probabilistic target rather than a modelling error.

The model is deployed as a validated Streamlit dashboard that handles missing sensor readings, reproduces the training predictions exactly and monitors incoming data with automatic review triggers. The most valuable next steps are to collect repeated measurements and real outcomes, so that trend features can be added and the threshold can be set from actual maintenance costs.

References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.

Markom, M. A. (2026). IFJC4002 Machine learning prediction project: Project brief and rubric. Faculty of Intelligent Computing, Universiti Malaysia Perlis.

IFJC4002 course team. (2026). 07_Machine_Failure.xlsx: Synthetic machine failure dataset for teaching [Data set]. Provided by the course lecturer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Streamlit Inc. (2026). Streamlit documentation. https://docs.streamlit.io

Appendix A: Contribution Record

All five members took part in the demonstration and can explain the code. The main responsibilities were:

Table 17: Contribution of each member

No.	Member (Student ID)	Main responsibility	Specific tasks and evidence
1	Wuttichok Samathi (67115675)	Project lead, deployment and monitoring	GitHub repository and commits; Streamlit Cloud deployment and Python-version fix; Colab notebook integration; monitoring triggers; report Section 6.4 and 6.6
2	Watsana Onbut (67128256)	Data analysis (Task 1)	Data quality checks, class balance, quintile and correlation analysis, EDA figures; report Section 2
3	Sunhanat Thongon (67123232)	Preprocessing pipeline	features.py, median imputation, engineered features, train/validation/test split and leakage control; report Section 3
4	Anas Waekaji (67134155)	Model development and evaluation (Task 2)	Model grids, cross-validation, threshold tuning, model comparison, error analysis, feature importance; report Sections 4 and 5
5	Ratthanan Yidnorradin (67106716)	Dashboard and testing (Task 3)	app.py, predictor.py input validation, automated tests, test log and screenshots; report Section 6.1 to 6.3 and 6.5

Appendix B: AI-Assisted Development Record

The group used an AI assistant (Claude by Anthropic) for coding support. Representative prompts from different development stages are listed below, translated from Thai, with how the output was checked and changed.

Table 18: Representative AI-assisted prompts and verification

No.	Stage	Prompt (translated)	AI output	How we checked and what we changed
1	Project scoping	Do Case 7 (project brief PDF attached).	Project structure, training script, Streamlit app, tests and report outline	Compared every file against the brief and rubric; later replaced the first public dataset with the lecturer dataset
2	Data analysis	Why are these values 0? Please check. (screenshots of describe and isna output)	Explanation that 0 missing values and zero quartiles of binary flags are expected	Confirmed class counts with value_counts; added a cell that stops Run all if training fails
3	Reproducible workflow	Make it a Google Colab notebook and send it.	Self-contained notebook that writes all files, trains, tests and zips the project	Ran the notebook end to end in Colab; moved the download step before the optional preview after it was not reached
4	Dataset change	Use this data (07_Machine_Failure.xlsx).	Code rewritten for 7 inputs, median imputation and a 60/20/20 split following the Guide sheet	Checked column meanings against the Guide sheet; verified with pytest and 15 of 15 agreement cases
5	Deployment	I have waited almost 10 minutes, is this normal? (build-log screenshot)	Diagnosis: Python 3.14 had no pre-built packages for the pinned libraries	Redeployed with the Colab Python version; the app built in about two minutes
6	Testing	Why do the numbers look strange? Check whether they are correct. (agreement tab)	Explanation that differences of about 1e-17 are floating-point rounding	Confirmed all cases match; changed the display to scientific notation with an explanatory caption
7	Maintenance	Give me code to save everything to Google Drive and link with GitHub.	Notebook cells for Drive backup and a GitHub push using a token from Colab Secrets	Kept the token out of the notebook; checked the repository after pushing

Appendix C: Test Log and Monitoring Record

Tests were run on the deployed application on 29 October 2026. Input order: air °C, process °C, rpm, torque N·m, tool wear min, vibration mm/s, age years.

Table 19: Test log of the deployed dashboard

ID	Input	Expected behaviour	Actual result	Pass
N1	28, 72.5, 1700, 50, 120, 2.4, 7.5	No Failure, no warning	NO FAILURE, 20.8%	Yes
N2	28, 72.5, 1700, 82, 245, 5.5, 14	Failure	FAILURE, 99.4%	Yes
B1	Speed 3,500 rpm (training max 2,499)	Prediction with extrapolation warning	Warning shown, prediction made	Yes
B2	Air temperature marked unavailable	Median 27.7 used, warning	Warning shown, prediction made (13.0%)	Yes
B3	All inputs at training minimum / maximum	No range warning; matches notebook	Matches notebook (0.0003 / 0.9963)	Yes
B4	Both temperatures missing (reference case)	Imputed; matches notebook	Matches notebook (0.2357)	Yes
I1	Torque -5 (batch)	Row rejected	REJECTED: outside the valid range 0 to 150	Yes
I2	Vibration abc (batch)	Row rejected	REJECTED: must be a number	Yes
I3	Tool wear blank (batch)	Row rejected	REJECTED: Tool wear (min) is required	Yes
I4	Air temperature blank (batch)	Accepted with warning	OK (warning), median used	Yes
A1	Notebook agreement tab	All reference cases match	15 of 15 match, max difference 1.11e-16	Yes
A2	pytest -v in Colab (29 tests)	All tests pass	Record the number passed from the Colab output	
M1	40 high-risk machines logged	Review trigger fires	3 triggers fired (failure rate, wear and vibration shift)	Yes

Monitoring record (29 October 2026): 63 predictions logged; window of the 50 most recent predictions; predicted failure rate 84.0% (training 15.9%); mean failure probability 83.3%; out-of-range inputs 8%; missing temperatures 2%. Triggers: predicted failure rate above twice the training rate, mean tool wear shifted +1.36 standard deviations, mean vibration shifted +1.55 standard deviations. Interpretation: the batch of high-risk machines was submitted on purpose to test monitoring, and the triggers responded as designed.

Appendix D: Dashboard Screenshots

Screenshots of every page of the deployed application and of the GitHub repository, taken on 29 October 2026.

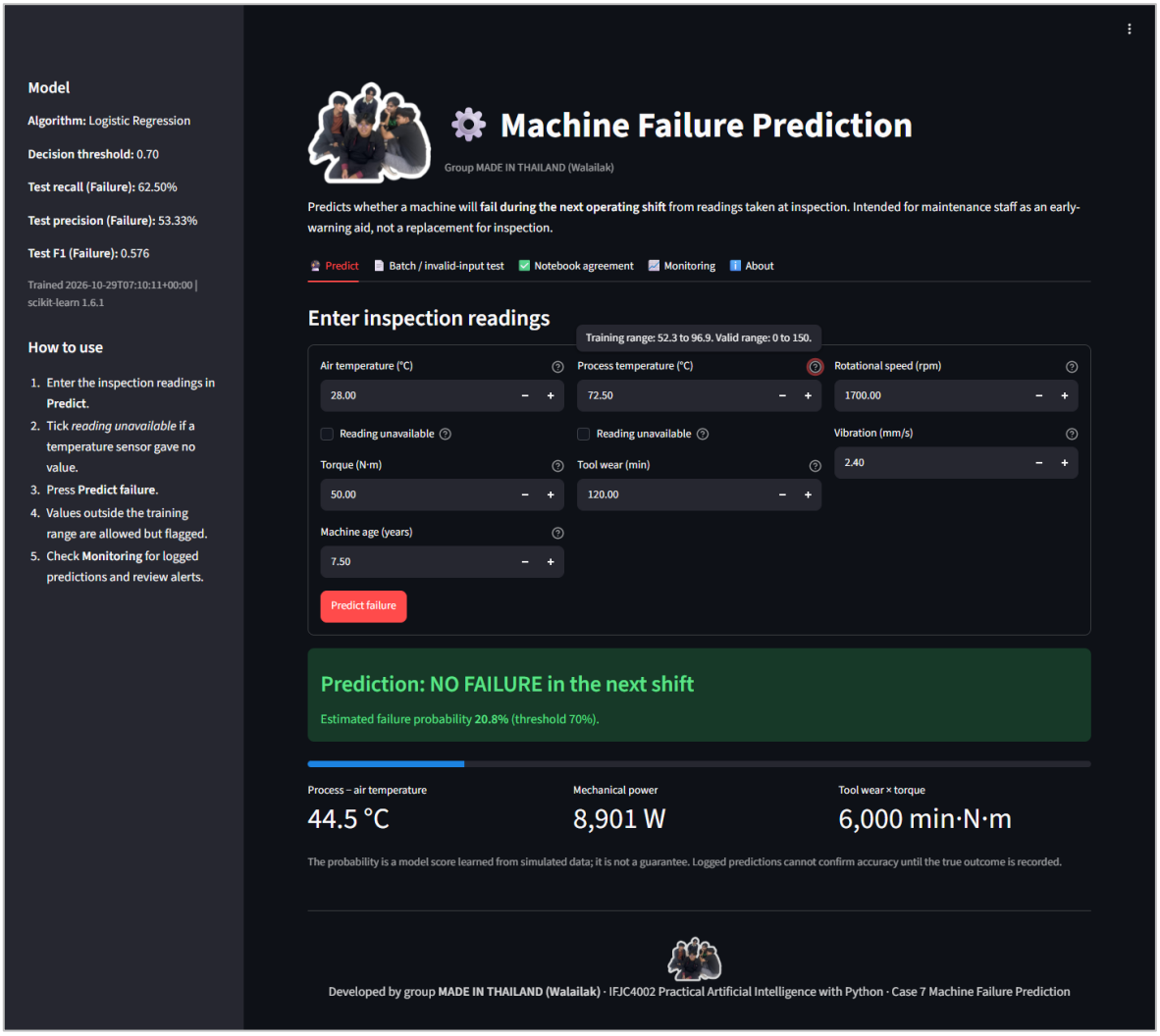


Figure 11: Predict tab: a typical machine is classified as NO FAILURE (probability 20.8%)

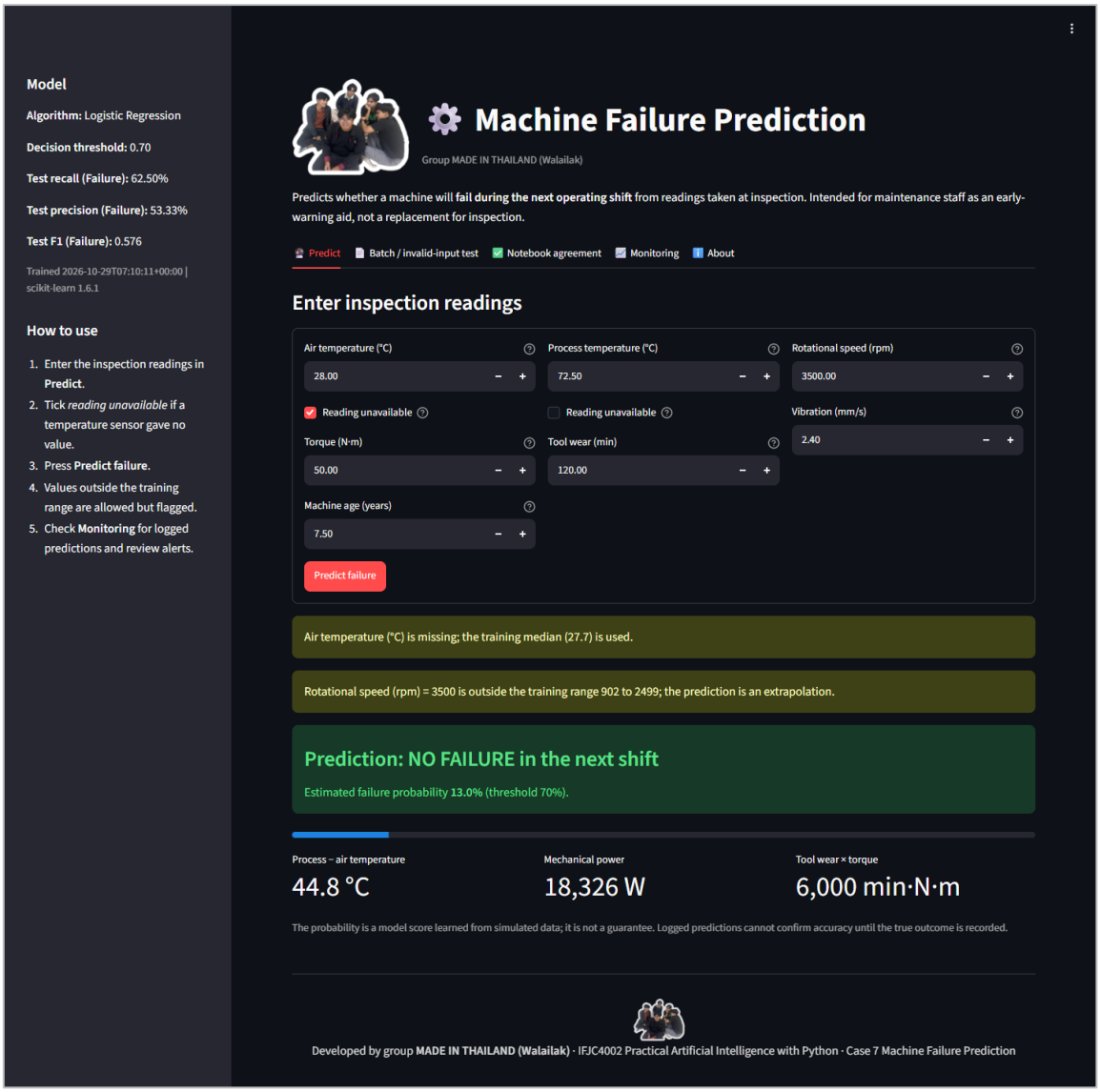


Figure 12: Predict tab: speed above the training range and a missing air temperature produce warnings

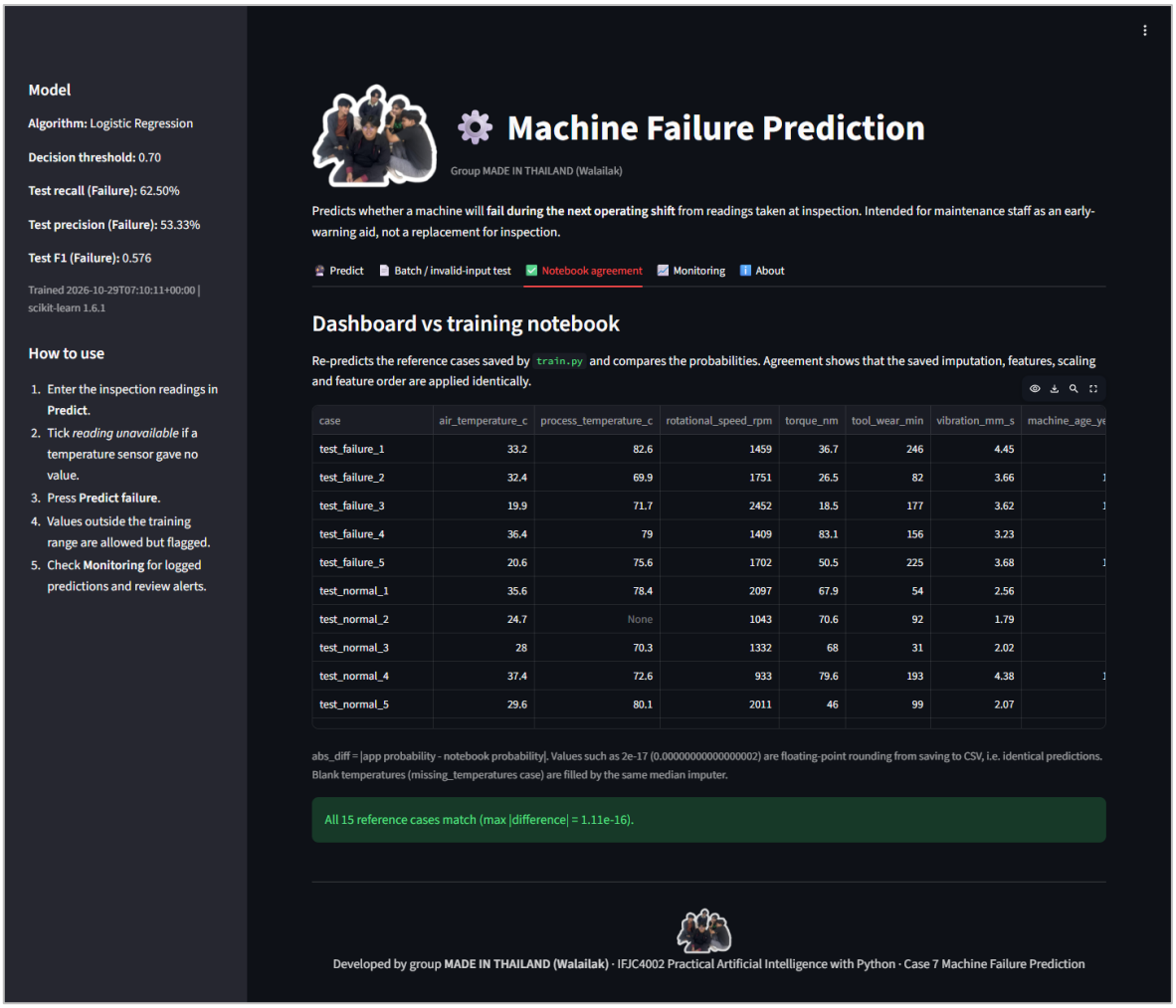


Figure 13: Notebook agreement tab: all 15 reference cases match the training script

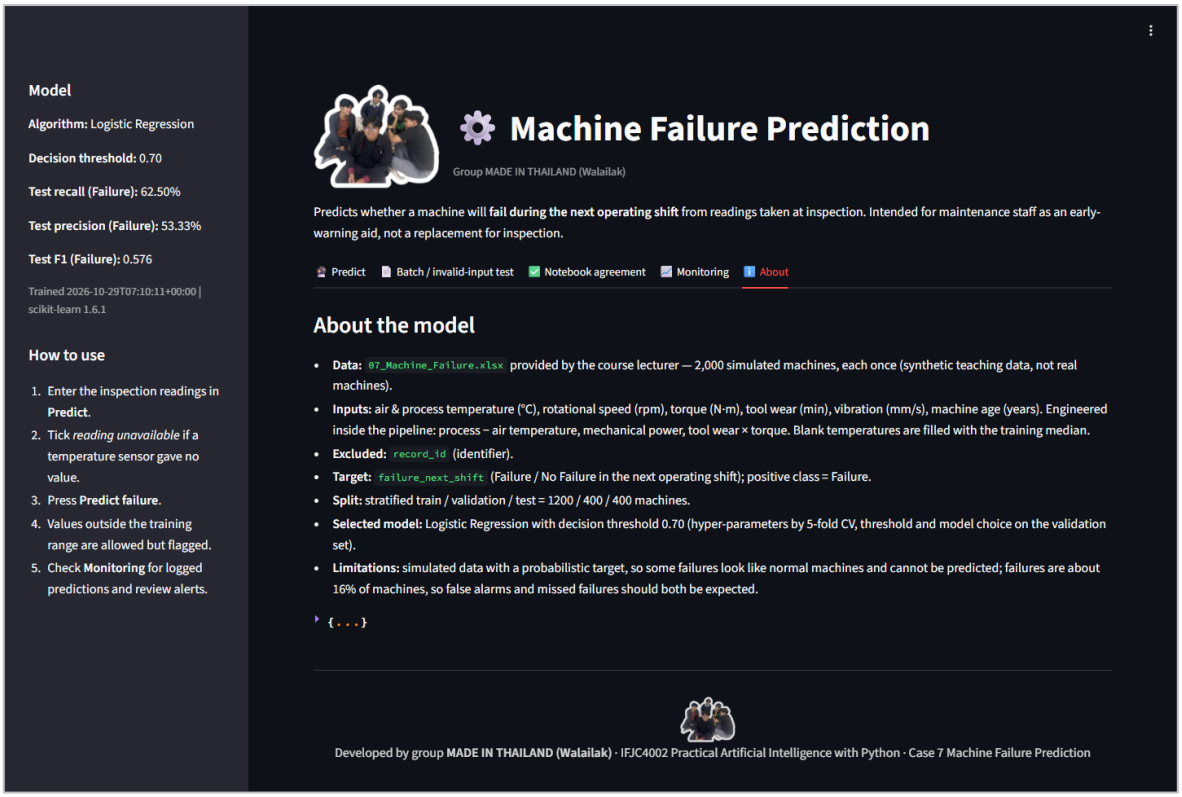


Figure 14: About tab: data, inputs, split, selected model and limitations

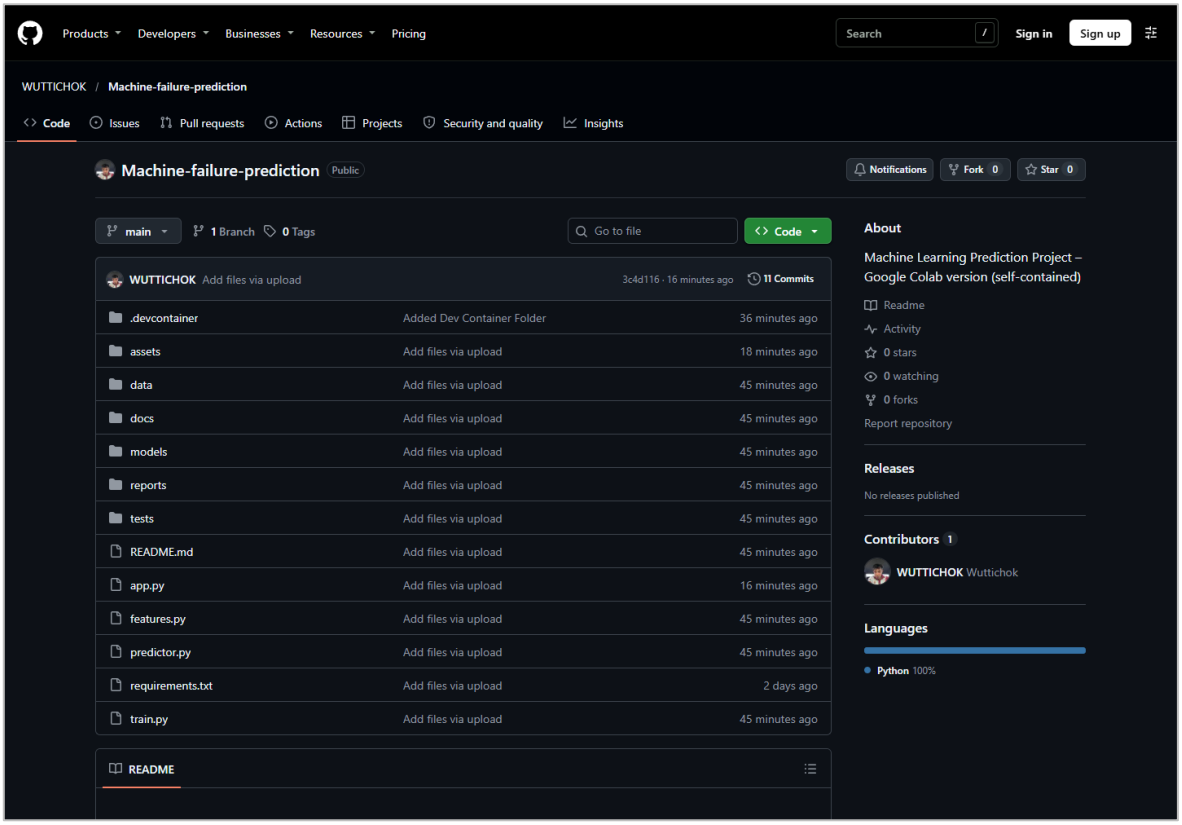


Figure 15: GitHub repository used for deployment

Appendix E: Additional Exploratory Figures

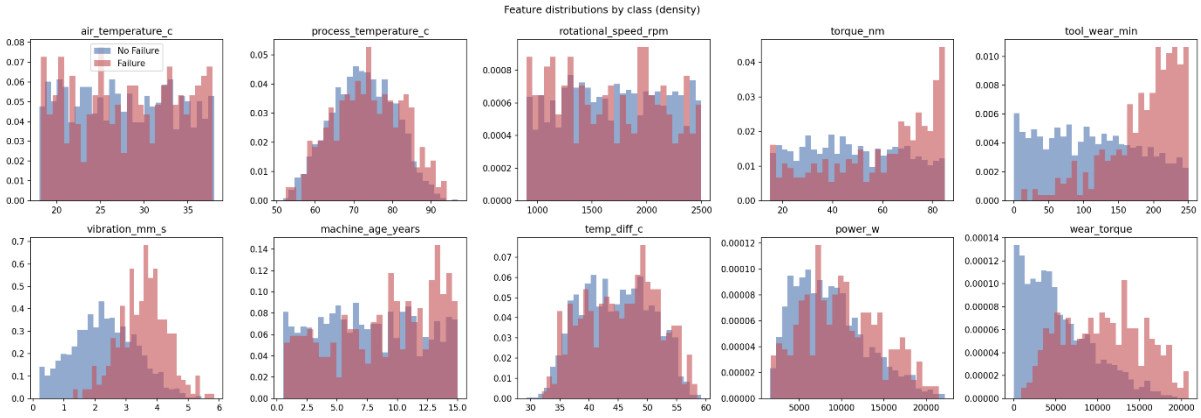


Figure 16: Distribution of each input and engineered feature by class

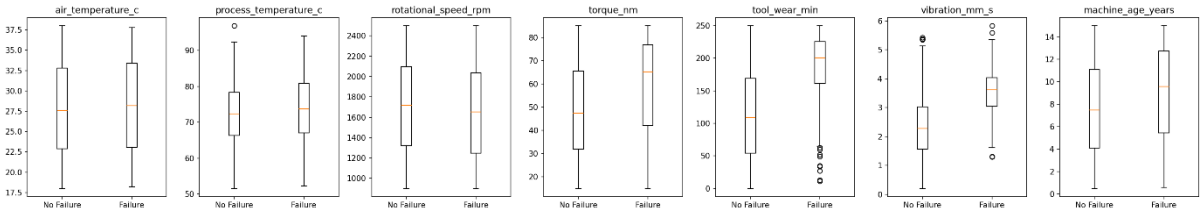


Figure 17: Box plots of the inputs by class

Reproduction: open Case7_Machine_Failure_Colab.ipynb in Google Colab, choose Runtime > Run all and upload 07_Machine_Failure.xlsx when asked. The notebook writes all project files, trains and evaluates the models, runs the tests, saves everything to Google Drive and can push the project to GitHub. Full instructions are in README.md.